

# Lab 4 Visualisation Pie Charts and Subplots

2026-01-16

## Week 4 Pie Charts and Subplots

### Goal

Create a Pie Chart to show the distribution of runs by Time of Day. Add into dashboard style visualisation.

### Imports

This week we are going to import all previous datasets.

```
library(dplyr)
library(plotly)
library(ggplot2)

file_id <- "1N1UERM9IGhJ29apnDalVLhx9EJBmvH-v"
url <- paste0("https://drive.google.com/uc?id=", file_id)

distancevpac <- read.csv(url)

file_id <- "1lgjURcPZaIDCqBa_bjbkmZk9i12M9x9W"
url <- paste0("https://drive.google.com/uc?id=", file_id)

trailing365 <- read.csv(url)

file_id <- "1MLylJ6cizJlTFVi_FdiF11pjpT3brls0"
url <- paste0("https://drive.google.com/uc?id=", file_id)

monthly_stats <- read.csv(url)

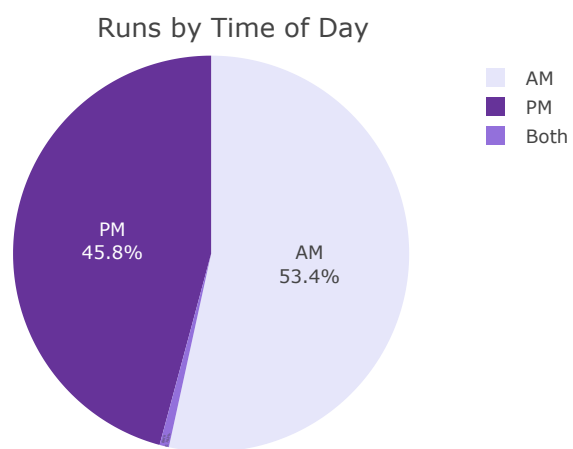
file_id <- "1K-PNkRuAPaVWP_fh8i0ZfJyulPvcK0ff"
url <- paste0("https://drive.google.com/uc?id=", file_id)

daily_summary <- read.csv(url)
```

### 1. Create a Pie Chart using Plotly or ggplotly.

Using `fig <- plot_ly`, you can set `type = "pie"` to render a pie chart. Be sure to set the titles and hover template.

<https://plotly.com/r/pie-charts/>



## 2. Put plots together

I've included code for the previous weeks' visualisations and converted them to Plotly where required. Using these (or your own plots), combine the charts into a single dashboard.

In Plotly this is usually straightforward using `subplot()`, where each plot is added as a new 2D cartesian trace in the final figure. There are two important things to note in this case:

- A pie chart does not use x/y axes, it is a domain chart. When combining it with other charts, you may need to manually specify its position to stop it overlapping other plots.
- When using `subplot()`, the final figure shares a single global layout, including titles and legends. For simplicity, it is recommended to disable the legend for now. We will explore adding per-plot titles using annotations in future weeks.

<https://plotly.com/r/subplots/>

```
#Pie Chart
pie_fig <- plot_ly(
  data = daily_summary,
  labels = ~AMPM,
  values = ~count,
  type = "pie",
  #Notice domain this specifies the position of the plot.
  domain = list(
    x = c(0.00, 0.48),
    y = c(0.52, 1.00)
  ),
  textinfo = "percent+label",
  textposition = "inside",
  hovertemplate = "%{label}: %{value}<extra></extra>",
  marker = list(colors = c("AM" = "#E6E6FA", "PM" = "rebeccapurple", "Both" = "mediumpurple"))[daily_summ
)

#Trailing 365 chart
trailing365$Date <- as.Date(trailing365$Date)
trailing365$hover_text <- paste0(
  "Date: ", trailing365$Date,
  "<br>Trailing365: ", sprintf("%.2f", trailing365$Trailing365)
)

p_trailing <- ggplot(trailing365, aes(x = Date, y = Trailing365, group = 1, text = hover_text)) +
  geom_line(colour = "purple") +
  labs(title = "Trailing 365-Day Kilometers",
    x = "Days", y = "Kilometers in past 365 days")

trail_fig <- ggplotly(p_trailing, tooltip = "text")

#Pace and Distance Scatter
p_scatter <- ggplot(distancevpace, aes(x = Distance_km, y = Pace_min_km)) +
  geom_point(colour = "purple") +
  labs(
    x = "Distance (km)",
    y = "Pace (min/km)"
  )
```

```

scatter_fig <- ggplotly(p_scatter)

#Bar Chart
bar_fig <- plot_ly(
  data = monthly_stats,
  x = ~YearMonth,
  y = ~(TotalDistance / 1000),
  type = "bar",
  marker = list(color = "rebeccapurple"),
  hovertemplate = paste(
    "Month: %{x}",
    "<br>Distance: %{y:.2f} km",
    "<extra></extra>"
  )
)

```

Running Dashboard

